

Phase 2: The Bento Cards (Adding interactive physics)

1. npm install gsap

Usage

```
import MagicBento from './MagicBento'
```

```
<MagicBento
```

```
  textAutoHide={true}
```

```
  enableStars
```

```
  enableSpotlight
```

```
  enableBorderGlow={true}
```

```
  enableTilt={false}
```

```
  enableMagnetism={false}
```

```
  clickEffect
```

```
  spotlightRadius={400}
```

```
  particleCount={12}
```

```
  glowColor="132, 0, 255"
```

```
  disableAnimations={false}
```

```
/>
```

```
import { useRef, useEffect, useCallback, useState } from 'react';
```

```
import { gsap } from 'gsap';
```

```
import './MagicBento.css';
```

```
const DEFAULT_PARTICLE_COUNT = 12;
```

```
const DEFAULT_SPOTLIGHT_RADIUS = 300;
```

```
const DEFAULT_GLOW_COLOR = '132, 0, 255';
```

```
const MOBILE_BREAKPOINT = 768;
```

```
const cardData = [
```

```
  {
```

```
    color: '#120F17',
```

```
    title: 'Analytics',
```

```
    description: 'Track user behavior',
    label: 'Insights'
  },
  {
    color: '#120F17',
    title: 'Dashboard',
    description: 'Centralized data view',
    label: 'Overview'
  },
  {
    color: '#120F17',
    title: 'Collaboration',
    description: 'Work together seamlessly',
    label: 'Teamwork'
  },
  {
    color: '#120F17',
    title: 'Automation',
    description: 'Streamline workflows',
    label: 'Efficiency'
  },
  {
    color: '#120F17',
    title: 'Integration',
    description: 'Connect favorite tools',
    label: 'Connectivity'
  },
  {
    color: '#120F17',
    title: 'Security',
    description: 'Enterprise-grade protection',
```

```
    label: 'Protection'
  }
];
```

```
const createParticleElement = (x, y, color = DEFAULT_GLOW_COLOR) => {
  const el = document.createElement('div');
  el.className = 'particle';
  el.style.cssText = `
    position: absolute;
    width: 4px;
    height: 4px;
    border-radius: 50%;
    background: rgba(${color}, 1);
    box-shadow: 0 0 6px rgba(${color}, 0.6);
    pointer-events: none;
    z-index: 100;
    left: ${x}px;
    top: ${y}px;
  `;
  return el;
};
```

```
const calculateSpotlightValues = radius => ({
  proximity: radius * 0.5,
  fadeDistance: radius * 0.75
});
```

```
const updateCardGlowProperties = (card, mouseX, mouseY, glow, radius) => {
  const rect = card.getBoundingClientRect();
  const relativeX = ((mouseX - rect.left) / rect.width) * 100;
  const relativeY = ((mouseY - rect.top) / rect.height) * 100;
```

```
card.style.setProperty('--glow-x', `${relativeX}%`);
card.style.setProperty('--glow-y', `${relativeY}%`);
card.style.setProperty('--glow-intensity', glow.toString());
card.style.setProperty('--glow-radius', `${radius}px`);
};
```

```
const ParticleCard = ({
  children,
  className = "",
  disableAnimations = false,
  style,
  particleCount = DEFAULT_PARTICLE_COUNT,
  glowColor = DEFAULT_GLOW_COLOR,
  enableTilt = true,
  clickEffect = false,
  enableMagnetism = false
}) => {
  const cardRef = useRef(null);
  const particlesRef = useRef([]);
  const timeoutsRef = useRef([]);
  const isHoveredRef = useRef(false);
  const memoizedParticles = useRef([]);
  const particlesInitialized = useRef(false);
  const magnetismAnimationRef = useRef(null);

  const initializeParticles = useCallback(() => {
    if (particlesInitialized.current || !cardRef.current) return;

    const { width, height } = cardRef.current.getBoundingClientRect();
    memoizedParticles.current = Array.from({ length: particleCount }, () =>
```

```
    createParticleElement(Math.random() * width, Math.random() * height, glowColor)
  );
  particlesInitialized.current = true;
}, [particleCount, glowColor]);
```

```
const clearAllParticles = useCallback(() => {
  timeoutsRef.current.forEach(clearTimeout);
  timeoutsRef.current = [];
  magnetismAnimationRef.current?.kill();
```

```
  particlesRef.current.forEach(particle => {
    gsap.to(particle, {
      scale: 0,
      opacity: 0,
      duration: 0.3,
      ease: 'back.in(1.7)',
      onComplete: () => {
        particle.parentNode?.removeChild(particle);
      }
    });
  });
  particlesRef.current = [];
}, []);
```

```
const animateParticles = useCallback(() => {
  if (!cardRef.current || !isHoveredRef.current) return;

  if (!particlesInitialized.current) {
    initializeParticles();
  }
}
```

```

memoizedParticles.current.forEach((particle, index) => {
  const timeoutId = setTimeout(() => {
    if (!isHoveredRef.current || !cardRef.current) return;

    const clone = particle.cloneNode(true);
    cardRef.current.appendChild(clone);
    particlesRef.current.push(clone);

    gsap.fromTo(clone, { scale: 0, opacity: 0 }, { scale: 1, opacity: 1, duration: 0.3, ease:
'back.out(1.7)' });

    gsap.to(clone, {
      x: (Math.random() - 0.5) * 100,
      y: (Math.random() - 0.5) * 100,
      rotation: Math.random() * 360,
      duration: 2 + Math.random() * 2,
      ease: 'none',
      repeat: -1,
      yoyo: true
    });

    gsap.to(clone, {
      opacity: 0.3,
      duration: 1.5,
      ease: 'power2.inOut',
      repeat: -1,
      yoyo: true
    });
  }, index * 100);

  timeoutsRef.current.push(timeoutId);

```

```

    });
  }, [initializeParticles]);

useEffect(() => {
  if (disableAnimations || !cardRef.current) return;

  const element = cardRef.current;

  const handleMouseEnter = () => {
    isHoveredRef.current = true;
    animateParticles();

    if (enableTilt) {
      gsap.to(element, {
        rotateX: 5,
        rotateY: 5,
        duration: 0.3,
        ease: 'power2.out',
        transformPerspective: 1000
      });
    }
  };

  const handleMouseLeave = () => {
    isHoveredRef.current = false;
    clearAllParticles();

    if (enableTilt) {
      gsap.to(element, {
        rotateX: 0,
        rotateY: 0,

```

```
    duration: 0.3,  
    ease: 'power2.out'  
  });  
}
```

```
if (enableMagnetism) {  
  gsap.to(element, {  
    x: 0,  
    y: 0,  
    duration: 0.3,  
    ease: 'power2.out'  
  });  
}  
};
```

```
const handleMouseMove = e => {  
  if (!enableTilt && !enableMagnetism) return;  
  
  const rect = element.getBoundingClientRect();  
  const x = e.clientX - rect.left;  
  const y = e.clientY - rect.top;  
  const centerX = rect.width / 2;  
  const centerY = rect.height / 2;  
  
  if (enableTilt) {  
    const rotateX = ((y - centerY) / centerY) * -10;  
    const rotateY = ((x - centerX) / centerX) * 10;  
  
    gsap.to(element, {  
      rotateX,  
      rotateY,  

```

```
    duration: 0.1,  
    ease: 'power2.out',  
    transformPerspective: 1000  
  });  
}
```

```
if (enableMagnetism) {  
  const magnetX = (x - centerX) * 0.05;  
  const magnetY = (y - centerY) * 0.05;  
  
  magnetismAnimationRef.current = gsap.to(element, {  
    x: magnetX,  
    y: magnetY,  
    duration: 0.3,  
    ease: 'power2.out'  
  });  
}  
};
```

```
const handleClick = e => {  
  if (!clickEffect) return;  
  
  const rect = element.getBoundingClientRect();  
  const x = e.clientX - rect.left;  
  const y = e.clientY - rect.top;  
  
  const maxDistance = Math.max(  
    Math.hypot(x, y),  
    Math.hypot(x - rect.width, y),  
    Math.hypot(x, y - rect.height),  
    Math.hypot(x - rect.width, y - rect.height)
```

```
);
```

```
const ripple = document.createElement('div');  
ripple.style.cssText = `  
  position: absolute;  
  width: ${maxDistance * 2}px;  
  height: ${maxDistance * 2}px;  
  border-radius: 50%;  
  background: radial-gradient(circle, rgba(${glowColor}, 0.4) 0%, rgba(${glowColor}, 0.2) 30%,  
transparent 70%);  
  left: ${x - maxDistance}px;  
  top: ${y - maxDistance}px;  
  pointer-events: none;  
  z-index: 1000;  
`;
```

```
element.appendChild(ripple);
```

```
gsap.fromTo(  
  ripple,  
  {  
    scale: 0,  
    opacity: 1  
  },  
  {  
    scale: 1,  
    opacity: 0,  
    duration: 0.8,  
    ease: 'power2.out',  
    onComplete: () => ripple.remove()  
  }  
)
```

```
);  
};
```

```
element.addEventListener('mouseenter', handleMouseEnter);  
element.addEventListener('mouseleave', handleMouseLeave);  
element.addEventListener('mousemove', handleMouseMove);  
element.addEventListener('click', handleClick);
```

```
return () => {  
  isHoveredRef.current = false;  
  element.removeEventListener('mouseenter', handleMouseEnter);  
  element.removeEventListener('mouseleave', handleMouseLeave);  
  element.removeEventListener('mousemove', handleMouseMove);  
  element.removeEventListener('click', handleClick);  
  clearAllParticles();  
};
```

```
}, [animateParticles, clearAllParticles, disableAnimations, enableTilt, enableMagnetism, clickEffect,  
glowColor]);
```

```
return (  
  <div  
    ref={cardRef}  
    className={` ${className} particle-container`}  
    style={{ ...style, position: 'relative', overflow: 'hidden' }}  
  >  
    {children}  
  </div>  
);  
};
```

```
const GlobalSpotlight = ({
```

```

gridRef,
disableAnimations = false,
enabled = true,
spotlightRadius = DEFAULT_SPOTLIGHT_RADIUS,
glowColor = DEFAULT_GLOW_COLOR
}) => {
  const spotlightRef = useRef(null);
  const isInsideSection = useRef(false);

  useEffect(() => {
    if (disableAnimations || !gridRef?.current || !enabled) return;

    const spotlight = document.createElement('div');
    spotlight.className = 'global-spotlight';
    spotlight.style.cssText = `
      position: fixed;
      width: 800px;
      height: 800px;
      border-radius: 50%;
      pointer-events: none;
      background: radial-gradient(circle,
        rgba(${glowColor}, 0.15) 0%,
        rgba(${glowColor}, 0.08) 15%,
        rgba(${glowColor}, 0.04) 25%,
        rgba(${glowColor}, 0.02) 40%,
        rgba(${glowColor}, 0.01) 65%,
        transparent 70%
      );
      z-index: 200;
      opacity: 0;
      transform: translate(-50%, -50%);

```

```

    mix-blend-mode: screen;
`;
document.body.appendChild(spotlight);
spotlightRef.current = spotlight;

const handleMouseMove = e => {
    if (!spotlightRef.current || !gridRef.current) return;

    const section = gridRef.current.closest('.bento-section');
    const rect = section?.getBoundingClientRect();
    const mouseInside =
        rect && e.clientX >= rect.left && e.clientX <= rect.right && e.clientY >= rect.top && e.clientY <=
        rect.bottom;

    isInsideSection.current = mouseInside || false;
    const cards = gridRef.current.querySelectorAll('.magic-bento-card');

    if (!mouseInside) {
        gsap.to(spotlightRef.current, {
            opacity: 0,
            duration: 0.3,
            ease: 'power2.out'
        });
        cards.forEach(card => {
            card.style.setProperty('--glow-intensity', '0');
        });
        return;
    }
}

const { proximity, fadeDistance } = calculateSpotlightValues(spotlightRadius);
let minDistance = Infinity;

```

```

cards.forEach(card => {
  const cardElement = card;
  const cardRect = cardElement.getBoundingClientRect();
  const centerX = cardRect.left + cardRect.width / 2;
  const centerY = cardRect.top + cardRect.height / 2;
  const distance =
    Math.hypot(e.clientX - centerX, e.clientY - centerY) - Math.max(cardRect.width,
cardRect.height) / 2;
  const effectiveDistance = Math.max(0, distance);

  minDistance = Math.min(minDistance, effectiveDistance);

  let glowIntensity = 0;
  if (effectiveDistance <= proximity) {
    glowIntensity = 1;
  } else if (effectiveDistance <= fadeDistance) {
    glowIntensity = (fadeDistance - effectiveDistance) / (fadeDistance - proximity);
  }

  updateCardGlowProperties(cardElement, e.clientX, e.clientY, glowIntensity, spotlightRadius);
});

gsap.to(spotlightRef.current, {
  left: e.clientX,
  top: e.clientY,
  duration: 0.1,
  ease: 'power2.out'
});

const targetOpacity =

```

```

minDistance <= proximity
  ? 0.8
  : minDistance <= fadeDistance
    ? ((fadeDistance - minDistance) / (fadeDistance - proximity)) * 0.8
    : 0;

gsap.to(spotlightRef.current, {
  opacity: targetOpacity,
  duration: targetOpacity > 0 ? 0.2 : 0.5,
  ease: 'power2.out'
});
};

const handleMouseLeave = () => {
  isInsideSection.current = false;
  gridRef.current?.querySelectorAll('.magic-bento-card').forEach(card => {
    card.style.setProperty('--glow-intensity', '0');
  });
  if (spotlightRef.current) {
    gsap.to(spotlightRef.current, {
      opacity: 0,
      duration: 0.3,
      ease: 'power2.out'
    });
  }
};

document.addEventListener('mousemove', handleMouseMove);
document.addEventListener('mouseleave', handleMouseLeave);

return () => {

```

```

    document.removeEventListener('mousemove', handleMouseMove);
    document.removeEventListener('mouseleave', handleMouseLeave);
    spotlightRef.current?.parentNode?.removeChild(spotlightRef.current);
  };
}, [gridRef, disableAnimations, enabled, spotlightRadius, glowColor]);

return null;
};

const BentoCardGrid = ({ children, gridRef }) => (
  <div className="card-grid bento-section" ref={gridRef}>
    {children}
  </div>
);

const useMobileDetection = () => {
  const [isMobile, setIsMobile] = useState(false);

  useEffect(() => {
    const checkMobile = () => setIsMobile(window.innerWidth <= MOBILE_BREAKPOINT);

    checkMobile();
    window.addEventListener('resize', checkMobile);

    return () => window.removeEventListener('resize', checkMobile);
  }, []);

  return isMobile;
};

const MagicBento = ({

```

```

textAutoHide = true,
enableStars = true,
enableSpotlight = true,
enableBorderGlow = true,
disableAnimations = false,
spotlightRadius = DEFAULT_SPOTLIGHT_RADIUS,
particleCount = DEFAULT_PARTICLE_COUNT,
enableTilt = false,
glowColor = DEFAULT_GLOW_COLOR,
clickEffect = true,
enableMagnetism = true
}) => {
  const gridRef = useRef(null);
  const isMobile = useMobileDetection();
  const shouldDisableAnimations = disableAnimations || isMobile;

  return (
    <>
    {enableSpotlight && (
      <GlobalSpotlight
        gridRef={gridRef}
        disableAnimations={shouldDisableAnimations}
        enabled={enableSpotlight}
        spotlightRadius={spotlightRadius}
        glowColor={glowColor}
      />
    )}

    <BentoCardGrid gridRef={gridRef}>
      {cardData.map((card, index) => {

```

```
const baseClassName = `magic-bento-card ${textAutoHide ? 'magic-bento-card--text-autohide' : ''} ${enableBorderGlow ? 'magic-bento-card--border-glow' : ''}`;
```

```
const cardProps = {  
  className: baseClassName,  
  style: {  
    backgroundColor: card.color,  
    '--glow-color': glowColor  
  }  
};
```

```
if (enableStars) {  
  return (  
    <ParticleCard  
      key={index}  
      {...cardProps}  
      disableAnimations={shouldDisableAnimations}  
      particleCount={particleCount}  
      glowColor={glowColor}  
      enableTilt={enableTilt}  
      clickEffect={clickEffect}  
      enableMagnetism={enableMagnetism}  
    >  
      <div className="magic-bento-card__header">  
        <div className="magic-bento-card__label">{card.label}</div>  
      </div>  
      <div className="magic-bento-card__content">  
        <h2 className="magic-bento-card__title">{card.title}</h2>  
        <p className="magic-bento-card__description">{card.description}</p>  
      </div>  
    </ParticleCard>  
  );  
}
```

```
}
```

```
return (
```

```
  <div
```

```
    key={index}
```

```
    {...cardProps}
```

```
    ref={el => {
```

```
      if (!el) return;
```

```
      const handleMouseMove = e => {
```

```
        if (shouldDisableAnimations) return;
```

```
        const rect = el.getBoundingClientRect();
```

```
        const x = e.clientX - rect.left;
```

```
        const y = e.clientY - rect.top;
```

```
        const centerX = rect.width / 2;
```

```
        const centerY = rect.height / 2;
```

```
        if (enableTilt) {
```

```
          const rotateX = ((y - centerY) / centerY) * -10;
```

```
          const rotateY = ((x - centerX) / centerX) * 10;
```

```
          gsap.to(el, {
```

```
            rotateX,
```

```
            rotateY,
```

```
            duration: 0.1,
```

```
            ease: 'power2.out',
```

```
            transformPerspective: 1000
```

```
          });
```

```
        }
```

```
        if (enableMagnetism) {
```

```
const magnetX = (x - centerX) * 0.05;
const magnetY = (y - centerY) * 0.05;

gsap.to(el, {
  x: magnetX,
  y: magnetY,
  duration: 0.3,
  ease: 'power2.out'
});
}
```

```
const handleMouseLeave = () => {
  if (shouldDisableAnimations) return;
```

```
  if (enableTilt) {
    gsap.to(el, {
      rotateX: 0,
      rotateY: 0,
      duration: 0.3,
      ease: 'power2.out'
    });
  }
```

```
  if (enableMagnetism) {
    gsap.to(el, {
      x: 0,
      y: 0,
      duration: 0.3,
      ease: 'power2.out'
    });
  }
```

```
};
```

```
const handleClick = e => {  
  if (!clickEffect || shouldDisableAnimations) return;
```

```
  const rect = el.getBoundingClientRect();
```

```
  const x = e.clientX - rect.left;
```

```
  const y = e.clientY - rect.top;
```

```
  const maxDistance = Math.max(  
    Math.hypot(x, y),  
    Math.hypot(x - rect.width, y),  
    Math.hypot(x, y - rect.height),  
    Math.hypot(x - rect.width, y - rect.height)  
  );
```

```
  const ripple = document.createElement('div');
```

```
  ripple.style.cssText = `
```

```
    position: absolute;
```

```
    width: ${maxDistance * 2}px;
```

```
    height: ${maxDistance * 2}px;
```

```
    border-radius: 50%;
```

```
    background: radial-gradient(circle, rgba(${glowColor}, 0.4) 0%, rgba(${glowColor}, 0.2)  
30%, transparent 70%);
```

```
    left: ${x - maxDistance}px;
```

```
    top: ${y - maxDistance}px;
```

```
    pointer-events: none;
```

```
    z-index: 1000;
```

```
`;
```

```
  el.appendChild(ripple);
```

```

gsap.fromTo(
  ripple,
  {
    scale: 0,
    opacity: 1
  },
  {
    scale: 1,
    opacity: 0,
    duration: 0.8,
    ease: 'power2.out',
    onComplete: () => ripple.remove()
  }
);
};

el.addEventListener('mousemove', handleMouseMove);
el.addEventListener('mouseleave', handleMouseLeave);
el.addEventListener('click', handleClick);
}}
>
<div className="magic-bento-card__header">
  <div className="magic-bento-card__label">{card.label}</div>
</div>
<div className="magic-bento-card__content">
  <h2 className="magic-bento-card__title">{card.title}</h2>
  <p className="magic-bento-card__description">{card.description}</p>
</div>
</div>
);

```

```

    }}}
  </BentoCardGrid>
</>
);
};

export default MagicBento;

:root {
  --hue: 27;
  --sat: 69%;
  --white: hsl(0, 0%, 100%);
  --purple-primary: rgba(132, 0, 255, 1);
  --purple-glow: rgba(132, 0, 255, 0.2);
  --purple-border: rgba(132, 0, 255, 0.8);
  --border-color: #2F293A;
  --background-dark: #120F17;
  color-scheme: light dark;
}

.card-grid {
  display: grid;
  gap: 0.5em;
  padding: 0.75em;
  max-width: 54em;
  font-size: clamp(1rem, 0.9rem + 0.5vw, 1.5rem);
}

.magic-bento-card {
  display: flex;
  flex-direction: column;

```

```
justify-content: space-between;
position: relative;
aspect-ratio: 4/3;
min-height: 200px;
width: 100%;
max-width: 100%;
padding: 1.25em;
border-radius: 20px;
border: 1px solid var(--border-color);
background: var(--background-dark);
font-weight: 300;
overflow: hidden;
transition: all 0.3s ease;
```

```
--glow-x: 50%;
--glow-y: 50%;
--glow-intensity: 0;
--glow-radius: 200px;
```

```
}
```

```
.magic-bento-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 25px rgba(0, 0, 0, 0.15);
}
```

```
.magic-bento-card__header,
.magic-bento-card__content {
  display: flex;
  position: relative;
  color: var(--white);
}
```

```
.magic-bento-card__header {  
  gap: 0.75em;  
  justify-content: space-between;  
}
```

```
.magic-bento-card__content {  
  flex-direction: column;  
}
```

```
.magic-bento-card__label {  
  font-size: 16px;  
}
```

```
.magic-bento-card__title,  
.magic-bento-card__description {  
  --clamp-title: 1;  
  --clamp-desc: 2;  
}
```

```
.magic-bento-card__title {  
  font-weight: 400;  
  font-size: 16px;  
  margin: 0 0 0.25em;  
}
```

```
.magic-bento-card__description {  
  font-size: 12px;  
  line-height: 1.2;  
  opacity: 0.9;  
}
```

```
.magic-bento-card--text-autohide .magic-bento-card__title,  
.magic-bento-card--text-autohide .magic-bento-card__description {  
  display: -webkit-box;  
  -webkit-box-orient: vertical;  
  overflow: hidden;  
  text-overflow: ellipsis;  
}
```

```
.magic-bento-card--text-autohide .magic-bento-card__title {  
  -webkit-line-clamp: var(--clamp-title);  
  line-clamp: var(--clamp-title);  
}
```

```
.magic-bento-card--text-autohide .magic-bento-card__description {  
  -webkit-line-clamp: var(--clamp-desc);  
  line-clamp: var(--clamp-desc);  
}
```

```
@media (max-width: 599px) {  
  .card-grid {  
    grid-template-columns: 1fr;  
    width: 90%;  
    margin: 0 auto;  
    padding: 0.5em;  
  }  
}
```

```
.magic-bento-card {  
  width: 100%;  
  min-height: 180px;  
}
```

```
}
```

```
@media (min-width: 600px) {  
  .card-grid {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

```
@media (min-width: 1024px) {  
  .card-grid {  
    grid-template-columns: repeat(4, 1fr);  
  }  
}
```

```
.magic-bento-card:nth-child(3) {  
  grid-column: span 2;  
  grid-row: span 2;  
}
```

```
.magic-bento-card:nth-child(4) {  
  grid-column: 1 / span 2;  
  grid-row: 2 / span 2;  
}
```

```
.magic-bento-card:nth-child(6) {  
  grid-column: 4;  
  grid-row: 3;  
}  
}
```

```
/* Border glow effect */
```

```
.magic-bento-card--border-glow::after {
```

```
content: "";
position: absolute;
inset: 0;
padding: 6px;
background: radial-gradient(
  var(--glow-radius) circle at var(--glow-x) var(--glow-y),
  rgba(132, 0, 255, calc(var(--glow-intensity) * 0.8)) 0%,
  rgba(132, 0, 255, calc(var(--glow-intensity) * 0.4)) 30%,
  transparent 60%
);
border-radius: inherit;
-webkit-mask:
  linear-gradient(#fff 0 0) content-box,
  linear-gradient(#fff 0 0);
-webkit-mask-composite: xor;
mask:
  linear-gradient(#fff 0 0) content-box,
  linear-gradient(#fff 0 0);
mask-composite: exclude;
pointer-events: none;
opacity: 1;
transition: opacity 0.3s ease;
z-index: 1;
}
```

```
.magic-bento-card--border-glow:hover::after {
  opacity: 1;
}
```

```
.magic-bento-card--border-glow:hover {
  box-shadow:
```

```
    0 4px 20px rgba(46, 24, 78, 0.4),
    0 0 30px var(--purple-glow);
}
```

```
.particle-container {
  position: relative;
  overflow: hidden;
}
```

```
.particle::before {
  content: "";
  position: absolute;
  top: -2px;
  left: -2px;
  right: -2px;
  bottom: -2px;
  background: rgba(132, 0, 255, 0.2);
  border-radius: 50%;
  z-index: -1;
}
```

```
.particle-container:hover {
  box-shadow:
    0 4px 20px rgba(46, 24, 78, 0.2),
    0 0 30px var(--purple-glow);
}
```

```
/* Global spotlight styles */
.global-spotlight {
  mix-blend-mode: screen;
  will-change: transform, opacity;
```

```
z-index: 200 !important;
pointer-events: none;
}
```

```
.bento-section {
  position: relative;
  user-select: none;
}
```

Phase 1: The Atmosphere (Replacing the flat background)

npm install ogl

Usage

```
import DarkVeil from './DarkVeil';
```

```
<div style={{ width: '100%', height: '600px', position: 'relative' }}>
  <DarkVeil
    hueShift={0}
    noiseIntensity={0}
    scanlineIntensity={0}
    speed={0.5}
    scanlineFrequency={0}
    warpAmount={0}
  />
</div>
```

Javascript

```
import { useRef, useEffect } from 'react';
import { Renderer, Program, Mesh, Triangle, Vec2 } from 'ogl';
import './DarkVeil.css';
```

```
const vertex = `
attribute vec2 position;
```

```
void main(){gl_Position=vec4(position,0.0,1.0);}

};
```

```
const fragment = `
```

```
#ifdef GL_ES
```

```
precision lowp float;
```

```
#endif
```

```
uniform vec2 uResolution;
```

```
uniform float uTime;
```

```
uniform float uHueShift;
```

```
uniform float uNoise;
```

```
uniform float uScan;
```

```
uniform float uScanFreq;
```

```
uniform float uWarp;
```

```
#define iTime uTime
```

```
#define iResolution uResolution
```

```
vec4 buf[8];
```

```
float rand(vec2 c){return fract(sin(dot(c,vec2(12.9898,78.233))))*43758.5453);}
```

```
mat3 rgb2yiq=mat3(0.299,0.587,0.114,0.596,-0.274,-0.322,0.211,-0.523,0.312);
```

```
mat3 yiq2rgb=mat3(1.0,0.956,0.621,1.0,-0.272,-0.647,1.0,-1.106,1.703);
```

```
vec3 hueShiftRGB(vec3 col,float deg){
```

```
    vec3 yiq=rgb2yiq*col;
```

```
    float rad=radians(deg);
```

```
    float cosh=cos(rad),sinh=sin(rad);
```

```
    vec3 yiqShift=vec3(yiq.x,yiq.y*cosh-yiq.z*sinh,yiq.y*sinh+yiq.z*cosh);
```

```
    return clamp(yiq2rgb*yiqShift,0.0,1.0);
```

```
}
```

```
vec4 sigmoid(vec4 x){return 1./(1.+exp(-x));}
```

```
vec4 cppn_fn(vec2 coordinate,float in0,float in1,float in2){
```

```
    buf[6]=vec4(coordinate.x,coordinate.y,0.3948333106474662+in0,0.36+in1);
```

```
    buf[7]=vec4(0.14+in2,sqrt(coordinate.x*coordinate.x+coordinate.y*coordinate.y),0.,0.);
```

```
    buf[0]=mat4(vec4(6.5404263,-3.6126034,0.7590882,-1.13613),vec4(2.4582713,3.1660357,1.2219609,0.06276096),vec4(-5.478085,-6.159632,1.8701609,-4.7742867),vec4(6.039214,-5.542865,-0.90925294,3.251348))*buf[6]+mat4(vec4(0.8473259,-5.722911,3.975766,1.6522468),vec4(-0.24321538,0.5839259,-1.7661959,-5.350116),vec4(0.,0.,0.,0.),vec4(0.,0.,0.,0.))*buf[7]+vec4(0.21808943,1.1243913,-1.7969975,5.0294676);
```

```
    buf[1]=mat4(vec4(-3.3522482,-6.0612736,0.55641043,-4.4719114),vec4(0.8631464,1.7432913,5.643898,1.6106541),vec4(2.4941394,-3.5012043,1.7184316,6.357333),vec4(3.310376,8.209261,1.1355612,-1.165539))*buf[6]+mat4(vec4(5.24046,-13.034365,0.009859298,15.870829),vec4(2.987511,3.129433,-0.89023495,-1.6822904),vec4(0.,0.,0.,0.),vec4(0.,0.,0.,0.))*buf[7]+vec4(-5.9457836,-6.573602,-0.8812491,1.5436668);
```

```
    buf[0]=sigmoid(buf[0]);buf[1]=sigmoid(buf[1]);
```

```
    buf[2]=mat4(vec4(-15.219568,8.095543,-2.429353,-1.9381982),vec4(-5.951362,4.3115187,2.6393783,1.274315),vec4(-7.3145227,6.7297835,5.2473326,5.9411426),vec4(5.0796127,8.979051,-1.7278991,-1.158976))*buf[6]+mat4(vec4(-11.967154,-11.608155,6.1486754,11.237008),vec4(2.124141,-6.263192,-1.7050359,-0.7021966),vec4(0.,0.,0.,0.),vec4(0.,0.,0.,0.))*buf[7]+vec4(-4.17164,-3.2281182,-4.576417,-3.6401186);
```

```
    buf[3]=mat4(vec4(3.1832156,-13.738922,1.879223,3.233465),vec4(0.64300746,12.768129,1.9141049,0.50990224),vec4(-0.049295485,4.4807224,1.4733979,1.801449),vec4(5.0039253,13.000481,3.3991797,-4.5561905))*buf[6]+mat4(vec4(-0.1285731,7.720628,-3.1425676,4.742367),vec4(0.6393625,3.714393,-0.8108378,-0.39174938),vec4(0.,0.,0.,0.),vec4(0.,0.,0.,0.))*buf[7]+vec4(-1.1811101,-21.621881,0.7851888,1.2329718);
```

```
    buf[2]=sigmoid(buf[2]);buf[3]=sigmoid(buf[3]);
```

```
    buf[4]=mat4(vec4(5.214916,-7.183024,2.7228765,2.6592617),vec4(-5.601878,-25.3591,4.067988,0.4602802),vec4(-10.57759,24.286327,21.102104,37.546658),vec4(4.3024497,-1.9625226,2.3458803,-1.372816))*buf[0]+mat4(vec4(-17.6526,-10.507558,2.2587414,12.462782),vec4(6.265566,-502.75443,-12.642513,0.9112289),vec4(-10.983244,20.741234,-9.701768,-0.7635988),vec4(5.383626,1.4819539,-4.1911616,-4.8444734))*buf[1]+mat4(vec4(12.785233,-16.345072,-0.39901125,1.7955981),vec4(-30.48365,-1.8345358,1.4542528,-1.1118771),vec4(19.872723,-7.337935,-42.941723,-98.52709),vec4(8.337645,-2.7312303,-2.2927687,-36.142323))*buf[2]+mat4(vec4(-16.298317,3.5471997,-0.44300047,-9.444417),vec4(57.5077,-35.609753,16.163465,-
```

4.1534753),vec4(-0.07470326,-3.8656476,-7.0901804,3.1523974),vec4(-12.559385,-
7.077619,1.490437,-0.8211543))*buf[3]+vec4(-7.67914,15.927437,1.3207729,-1.6686112);

buf[5]=mat4(vec4(-1.4109162,-0.372762,-3.770383,-21.367174),vec4(-6.2103205,-
9.35908,0.92529047,8.82561),vec4(11.460242,-22.348068,13.625772,-18.693201),vec4(-
0.3429052,-3.9905605,-2.4626114,-0.45033523))*buf[0]+mat4(vec4(7.3481627,-4.3661838,-
6.3037653,-3.868115),vec4(1.5462853,6.5488915,1.9701879,-0.58291394),vec4(6.5858274,-
2.2180402,3.7127688,-1.3730392),vec4(-5.7973905,10.134961,-2.3395722,-
5.965605))*buf[1]+mat4(vec4(-2.5132585,-6.6685553,-1.4029363,-0.16285264),vec4(-
0.37908727,0.53738135,4.389061,-1.3024765),vec4(-0.70647055,2.0111287,-5.1659346,-
3.728635),vec4(-13.562562,10.487719,-0.9173751,-2.6487076))*buf[2]+mat4(vec4(-
8.645013,6.5546675,-6.3944063,-5.5933375),vec4(-0.57783127,-
1.077275,36.91025,5.736769),vec4(14.283112,3.7146652,7.1452246,-
4.5958776),vec4(2.7192075,3.6021907,-4.366337,-2.3653464))*buf[3]+vec4(-5.9000807,-
4.329569,1.2427121,8.59503);

buf[4]=sigmoid(buf[4]);buf[5]=sigmoid(buf[5]);

buf[6]=mat4(vec4(-1.61102,0.7970257,1.4675229,0.20917463),vec4(-28.793737,-
7.1390953,1.5025433,4.656581),vec4(-10.94861,39.66238,0.74318546,-10.095605),vec4(-
0.7229728,-1.5483948,0.7301322,2.1687684))*buf[0]+mat4(vec4(3.2547753,21.489103,-
1.0194173,-3.3100595),vec4(-3.7316632,-3.3792162,-7.223193,-
0.23685838),vec4(13.1804495,0.7916005,5.338587,5.687114),vec4(-4.167605,-17.798311,-
6.815736,-1.6451967))*buf[1]+mat4(vec4(0.604885,-7.800309,-7.213122,-2.741014),vec4(-
3.522382,-0.12359311,-0.5258442,0.43852118),vec4(9.6752825,-
22.853785,2.062431,0.099892326),vec4(-4.3196306,-
17.730087,2.5184598,5.30267))*buf[2]+mat4(vec4(-6.545563,-15.790176,-6.0438633,-
5.415399),vec4(-43.591583,28.551912,-
16.00161,18.84728),vec4(4.212382,8.394307,3.0958717,8.657522),vec4(-5.0237565,-4.450633,-
4.4768,-5.5010443))*buf[3]+mat4(vec4(1.6985557,-
67.05806,6.897715,1.9004834),vec4(1.8680354,2.3915145,2.5231109,4.081538),vec4(11.158006,1.
7294737,2.0738268,7.386411),vec4(-4.256034,-306.24686,8.258898,-
17.132736))*buf[4]+mat4(vec4(1.6889864,-4.5852966,3.8534803,-6.3482175),vec4(1.3543309,-
1.2640043,9.932754,2.9079645),vec4(-5.2770967,0.07150358,-
0.13962056,3.3269649),vec4(28.34703,-
4.918278,6.1044083,4.085355))*buf[5]+vec4(6.6818056,12.522166,-3.7075126,-4.104386);

buf[7]=mat4(vec4(-8.265602,-4.7027016,5.098234,0.7509808),vec4(8.6507845,-
17.15949,16.51939,-8.884479),vec4(-4.036479,-2.3946867,-2.6055532,-1.9866527),vec4(-
2.2167742,-1.8135649,-5.9759874,4.8846445))*buf[0]+mat4(vec4(6.7790847,3.5076547,-
2.8191125,-2.7028968),vec4(-5.743024,-0.27844876,1.4958696,-
5.0517144),vec4(13.122226,15.735168,-2.9397483,-4.101023),vec4(-14.375265,-5.030483,-
6.2599335,2.9848232))*buf[1]+mat4(vec4(4.0950394,-0.94011575,-
5.674733,4.755022),vec4(4.3809423,4.8310084,1.7425908,-3.437416),vec4(2.117492,0.16342592,-
104.56341,16.949184),vec4(-5.22543,-2.994248,3.8350096,-1.9364246))*buf[2]+mat4(vec4(-
5.900337,1.7946124,-13.604192,-
3.8060522),vec4(6.6583457,31.911177,25.164474,91.81147),vec4(11.840538,4.1503043,-
0.7314397,6.768467),vec4(-6.3967767,4.034772,6.1714606,-
0.32874924))*buf[3]+mat4(vec4(3.4992442,-196.91893,-8.923708,2.8142626),vec4(3.4806502,-
3.1846354,5.1725626,5.1804223),vec4(-2.4009497,15.585794,1.2863957,2.0252278),vec4(-

```

71.25271,-62.441242,-8.138444,0.50670296))*buf[4]+mat4(vec4(-12.291733,-11.176166,-
7.3474145,4.390294),vec4(10.805477,5.6337385,-0.9385842,-4.7348723),vec4(-12.869276,-
7.039391,5.3029537,7.5436664),vec4(1.4593618,8.91898,3.5101583,5.840625))*buf[5]+vec4(2.241
5268,-6.705987,-0.98861027,-2.117676);

buf[6]=sigmoid(buf[6]);buf[7]=sigmoid(buf[7]);

buf[0]=mat4(vec4(1.6794263,1.3817469,2.9625452,0.),vec4(-1.8834411,-1.4806935,-
3.5924516,0.),vec4(-1.3279216,-1.0918057,-
2.3124623,0.),vec4(0.2662234,0.23235129,0.44178495,0.))*buf[0]+mat4(vec4(-0.6299101,-
0.5945583,-0.9125601,0.),vec4(0.17828953,0.18300213,0.18182953,0.),vec4(-2.96544,-2.5819945,-
4.9001055,0.),vec4(1.4195864,1.1868085,2.5176322,0.))*buf[1]+mat4(vec4(-1.2584374,-
1.0552157,-2.1688404,0.),vec4(-0.7200217,-0.52666044,-
1.438251,0.),vec4(0.15345335,0.15196142,0.272854,0.),vec4(0.945728,0.8861938,1.2766753,0.))*b
uf[2]+mat4(vec4(-2.4218085,-1.968602,-4.35166,0.),vec4(-22.683098,-18.0544,-
41.954372,0.),vec4(0.63792,0.5470648,1.1078634,0.),vec4(-1.5489894,-1.3075932,-
2.6444845,0.))*buf[3]+mat4(vec4(-0.49252132,-0.39877754,-
0.91366625,0.),vec4(0.95609266,0.7923952,1.640221,0.),vec4(0.30616966,0.15693925,0.8639857,0
.),vec4(1.1825981,0.94504964,2.176963,0.))*buf[4]+mat4(vec4(0.35446745,0.3293795,0.59547555,
0.),vec4(-0.58784515,-0.48177817,-
1.0614829,0.),vec4(2.5271258,1.9991658,4.6846647,0.),vec4(0.13042648,0.08864098,0.30187556,0
.))*buf[5]+mat4(vec4(-1.7718065,-1.4033192,-
3.3355875,0.),vec4(3.1664357,2.638297,5.378702,0.),vec4(-3.1724713,-2.6107926,-
5.549295,0.),vec4(-2.851368,-2.249092,-
5.3013067,0.))*buf[6]+mat4(vec4(1.5203838,1.2212278,2.8404984,0.),vec4(1.5210563,1.2651345,2.
683903,0.),vec4(2.9789467,2.4364579,5.2347264,0.),vec4(2.2270417,1.8825914,3.8028636,0.))*buf
[7]+vec4(-1.5468478,-3.6171484,0.24762098,0.);

buf[0]=sigmoid(buf[0]);

return vec4(buf[0].x,buf[0].y,buf[0].z,1.);
}

```

```

void mainImage(out vec4 fragColor,in vec2 fragCoord){

vec2 uv=fragCoord/uResolution.xy*2.-1.;

uv.y*=-1.;

uv+=uWarp*vec2(sin(uv.y*6.283+uTime*0.5),cos(uv.x*6.283+uTime*0.5))*0.05;

fragColor=cppn_fn(uv,0.1*sin(0.3*uTime),0.1*sin(0.69*uTime),0.1*sin(0.44*uTime));

}

```

```

void main(){

vec4 col;mainImage(col,gl_FragCoord.xy);

col.rgb=hueShiftRGB(col.rgb,uHueShift);

```

```

float scanline_val=sin(gl_FragCoord.y*uScanFreq)*0.5+0.5;
col.rgb*=1.-(scanline_val*scanline_val)*uScan;
col.rgb+=(rand(gl_FragCoord.xy+uTime)-0.5)*uNoise;
gl_FragColor=vec4(clamp(col.rgb,0.0,1.0),1.0);
}
`;

```

```

export default function DarkVeil({
  hueShift = 0,
  noiseIntensity = 0,
  scanlineIntensity = 0,
  speed = 0.5,
  scanlineFrequency = 0,
  warpAmount = 0,
  resolutionScale = 1
}) {
  const ref = useRef(null);
  useEffect(() => {
    const canvas = ref.current;
    const parent = canvas.parentElement;

    const renderer = new Renderer({
      dpr: Math.min(window.devicePixelRatio, 2),
      canvas
    });

    const gl = renderer.gl;
    const geometry = new Triangle(gl);

    const program = new Program(gl, {
      vertex,

```

```
fragment,  
uniforms: {  
  uTime: { value: 0 },  
  uResolution: { value: new Vec2() },  
  uHueShift: { value: hueShift },  
  uNoise: { value: noiseIntensity },  
  uScan: { value: scanlineIntensity },  
  uScanFreq: { value: scanlineFrequency },  
  uWarp: { value: warpAmount }  
}  
});
```

```
const mesh = new Mesh(gl, { geometry, program });
```

```
const resize = () => {  
  const w = parent.clientWidth,  
    h = parent.clientHeight;  
  renderer.setSize(w * resolutionScale, h * resolutionScale);  
  program.uniforms.uResolution.value.set(w, h);  
};
```

```
window.addEventListener('resize', resize);  
resize();
```

```
const start = performance.now();  
let frame = 0;
```

```
const loop = () => {  
  program.uniforms.uTime.value = ((performance.now() - start) / 1000) * speed;  
  program.uniforms.uHueShift.value = hueShift;  
  program.uniforms.uNoise.value = noiseIntensity;
```

```

    program.uniforms.uScan.value = scanlineIntensity;
    program.uniforms.uScanFreq.value = scanlineFrequency;
    program.uniforms.uWarp.value = warpAmount;
    renderer.render({ scene: mesh });
    frame = requestAnimationFrame(loop);
  };

  loop();

  return () => {
    cancelAnimationFrame(frame);
    window.removeEventListener('resize', resize);
  };
}, [hueShift, noiseIntensity, scanlineIntensity, speed, scanlineFrequency, warpAmount,
resolutionScale]);

return <canvas ref={ref} className="darkveil-canvas" />;
}

```

Css

```

.darkveil-canvas {
  width: 100%;
  height: 100%;
  display: block;
}

```

Phase 3: Cinematic Typography (Killing the static text)

Integrate the <GradientText /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: GradientText

Variant: JavaScript + CSS

Usage Example

```
```jsx
```

```
import GradientText from './GradientText'
```

```
// For a smoother animation, the gradient should start and end with the same color
```

```
<GradientText
```

```
 colors={['#40ffaa', '#4079ff', '#40ffaa', '#4079ff', '#40ffaa']}
```

```
 animationSpeed={3}
```

```
 showBorder={false}
```

```
 className="custom-class"
```

```
>
```

```
 Add a splash of color!
```

```
</GradientText>
```

```
```
```

Props

| Prop | Type | Default | Description |
|------|------|---------|-------------|
|------|------|---------|-------------|

| | | | |
|-------|-------|-------|-------|
| ----- | ----- | ----- | ----- |
|-------|-------|-------|-------|

| | | | |
|----------|-----------|---|---|
| children | ReactNode | - | The content to be displayed inside the gradient text. |
|----------|-----------|---|---|

| | | | |
|-----------|--------|----|---|
| className | string | '' | Adds custom classes to the root element for additional styling. |
|-----------|--------|----|---|

| | | | |
|--------|----------|-----------------------------------|--|
| colors | string[] | ['#5227FF', '#FF9FFC', '#B497CF'] | Array of colors for the gradient effect. |
|--------|----------|-----------------------------------|--|

| | | | |
|----------------|--------|---|---|
| animationSpeed | number | 8 | Duration of one animation cycle in seconds. |
|----------------|--------|---|---|

| | | | |
|-----------|--|--------------|--------------------------------------|
| direction | 'horizontal' 'vertical' 'diagonal' | 'horizontal' | Direction of the gradient animation. |
|-----------|--|--------------|--------------------------------------|

| | | | |
|--------------|---------|-------|---|
| pauseOnHover | boolean | false | Pauses the animation when hovering over the text. |
|--------------|---------|-------|---|

| | | | |
|------|---------|------|---|
| yoyo | boolean | true | Reverses animation direction at the end instead of looping. |
|------|---------|------|---|

| | | | |
|------------|---------|-------|---|
| showBorder | boolean | false | Displays a gradient border around the text. |
|------------|---------|-------|---|

Full Component Source

```
``jsx

import { useState, useCallback, useEffect, useRef } from 'react';
import { motion, useMotionValue, useAnimationFrame, useTransform } from 'motion/react';
import './GradientText.css';

export default function GradientText({
  children,
  className = '',
  colors = ['#5227FF', '#FF9FFC', '#B497CF'],
  animationSpeed = 8,
  showBorder = false,
  direction = 'horizontal',
  pauseOnHover = false,
  yoyo = true
}) {
  const [isPaused, setIsPaused] = useState(false);
  const progress = useMotionValue(0);
  const elapsedRef = useRef(0);
  const lastTimeRef = useRef(null);

  const animationDuration = animationSpeed * 1000;

  useAnimationFrame(time => {
    if (isPaused) {
      lastTimeRef.current = null;
      return;
    }

    if (lastTimeRef.current === null) {
      lastTimeRef.current = time;
    }
  });
}
```

```

    return;
}

const deltaTime = time - lastTimeRef.current;
lastTimeRef.current = time;
elapsedRef.current += deltaTime;

if (yoyo) {
    const fullCycle = animationDuration * 2;
    const cycleTime = elapsedRef.current % fullCycle;

    if (cycleTime < animationDuration) {
        progress.set((cycleTime / animationDuration) * 100);
    } else {
        progress.set(100 - ((cycleTime - animationDuration) / animationDuration) * 100);
    }
} else {
    // Continuously increase position for seamless looping
    progress.set((elapsedRef.current / animationDuration) * 100);
}
});

useEffect(() => {
    elapsedRef.current = 0;
    progress.set(0);
}, [animationSpeed, progress, yoyo]);

const backgroundPosition = useTransform(progress, p => {
    if (direction === 'horizontal') {
        return `${p}% 50%`;
    } else if (direction === 'vertical') {

```

```

    return `50% ${p}%`;
  } else {
    // For diagonal, move only horizontally to avoid interference patterns
    return `${p}% 50%`;
  }
});

```

```

const handleMouseEnter = useCallback(() => {
  if (pauseOnHover) setIsPaused(true);
}, [pauseOnHover]);

```

```

const handleMouseLeave = useCallback(() => {
  if (pauseOnHover) setIsPaused(false);
}, [pauseOnHover]);

```

```

const gradientAngle =
  direction === 'horizontal' ? 'to right' : direction === 'vertical' ? 'to bottom' : 'to bottom right';
// Duplicate first color at the end for seamless looping
const gradientColors = [...colors, colors[0]].join(', ');

```

```

const gradientStyle = {
  backgroundImage: `linear-gradient(${gradientAngle}, ${gradientColors})`,
  backgroundSize: direction === 'horizontal' ? '300% 100%' : direction === 'vertical' ? '100% 300%' :
'300% 300%',
  backgroundRepeat: 'repeat'
};

```

```

return (
  <motion.div
    className={`animated-gradient-text ${showBorder ? 'with-border' : ''} ${className}`}
    onMouseEnter={handleMouseEnter}

```

```

      onMouseLeave={handleMouseLeave}
    >
      {showBorder && <motion.div className="gradient-overlay" style={{ ...gradientStyle,
backgroundPosition }} />}
      <motion.div className="text-content" style={{ ...gradientStyle, backgroundPosition }}>
        {children}
      </motion.div>
    </motion.div>
  );
}

```

...

Component CSS

```

``css
.animated-gradient-text {
  position: relative;
  margin: 0 auto;
  display: flex;
  max-width: fit-content;
  flex-direction: row;
  align-items: center;
  justify-content: center;
  border-radius: 1.25rem;
  font-weight: 500;
  backdrop-filter: blur(10px);
  transition: box-shadow 0.5s ease-out;
  overflow: hidden;
  cursor: pointer;
}

```

```
.animated-gradient-text.with-border {  
  padding: 0.35rem 0.75rem;  
}
```

```
.gradient-overlay {  
  position: absolute;  
  top: 0;  
  left: 0;  
  right: 0;  
  bottom: 0;  
  border-radius: inherit;  
  z-index: 0;  
  pointer-events: none;  
}
```

```
.gradient-overlay::before {  
  content: "";  
  position: absolute;  
  left: 0;  
  top: 0;  
  border-radius: inherit;  
  width: calc(100% - 2px);  
  height: calc(100% - 2px);  
  left: 50%;  
  top: 50%;  
  transform: translate(-50%, -50%);  
  background-color: #120F17;  
  z-index: -1;  
}
```

```
.text-content {
```

```
display: inline-block;
position: relative;
z-index: 2;
background-clip: text;
-webkit-background-clip: text;
color: transparent;
}
```

...

Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

Phase 4: Magnetic Controls (Tactile buttons)

Integrate the <BorderGlow /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: BorderGlow

Variant: JavaScript + CSS

Usage Example

```
```jsx
```

```
import BorderGlow from './BorderGlow';
```

```

<BorderGlow
 edgeSensitivity={30}
 glowColor="40 80 80"
 backgroundColor="#120F17"
 borderRadius={28}
 glowRadius={40}
 glowIntensity={1.0}
 coneSpread={25}
 animated={false}
 colors={['#c084fc', '#f472b6', '#38bdf8']}
>
 <div style={{ padding: '2em' }}>
 <h2>Your Content Here</h2>
 <p>Hover near the edges to see the glow.</p>
 </div>
</BorderGlow>
...

```

### ### Props

Prop	Type	Default	Description
children	ReactNode	-	Content rendered inside the card.
className	string	""	Additional CSS classes for the outer wrapper.
edgeSensitivity	number	30	How close the pointer must be to the edge for the glow to appear (0-100).
glowColor	string	"40 80 80"	HSL values for the glow color, as "H S L" (e.g. "40 80 80").
backgroundColor	string	"#120F17"	Background color of the card.
borderRadius	number	28	Corner radius of the card in pixels.
glowRadius	number	40	How far the outer glow extends beyond the card in pixels.
glowIntensity	number	1.0	Multiplier for glow opacity (0.1-3.0).

| coneSpread | number | 25 | Width of the directional cone mask as a percentage (5-45). |

| animated | boolean | false | Play an intro sweep animation on mount. |

| colors | string[] | [...] | Array of 3 hex colors for the mesh gradient border, distributed across positions. |

### ### Full Component Source

```
``jsx
import { useRef, useCallback, useEffect } from 'react';
import './BorderGlow.css';

function parseHSL(hslStr) {
 const match = hslStr.match(/([\d.]+\s*)s*([\d.]+\s*)%?\s*([\d.]+\s*)%?/);
 if (!match) return { h: 40, s: 80, l: 80 };
 return { h: parseFloat(match[1]), s: parseFloat(match[2]), l: parseFloat(match[3]) };
}

function buildGlowVars(glowColor, intensity) {
 const { h, s, l } = parseHSL(glowColor);
 const base = `${h}deg ${s}% ${l}%`;
 const opacities = [100, 60, 50, 40, 30, 20, 10];
 const keys = ['', '-60', '-50', '-40', '-30', '-20', '-10'];
 const vars = {};
 for (let i = 0; i < opacities.length; i++) {
 vars[`--glow-color${keys[i]}`] = `hsl(${base} / ${Math.min(opacities[i] * intensity, 100)}%)`;
 }
 return vars;
}

const GRADIENT_POSITIONS = ['80% 55%', '69% 34%', '8% 6%', '41% 38%', '86% 85%', '82% 18%', '51% 4%'];

const GRADIENT_KEYS = ['--gradient-one', '--gradient-two', '--gradient-three', '--gradient-four', '--gradient-five', '--gradient-six', '--gradient-seven'];
```

```
const COLOR_MAP = [0, 1, 2, 0, 1, 2, 1];
```

```
function buildGradientVars(colors) {
 const vars = {};
 for (let i = 0; i < 7; i++) {
 const c = colors[Math.min(COLOR_MAP[i], colors.length - 1)];
 vars[GRADIENT_KEYS[i]] = `radial-gradient(at ${GRADIENT_POSITIONS[i]}, ${c} 0px, transparent 50%)`;
 }
 vars['--gradient-base'] = `linear-gradient(${colors[0]} 0 100%)`;
 return vars;
}
```

```
function easeOutCubic(x) { return 1 - Math.pow(1 - x, 3); }
```

```
function easeInCubic(x) { return x * x * x; }
```

```
function animateValue({ start = 0, end = 100, duration = 1000, delay = 0, ease = easeOutCubic, onUpdate, onEnd }) {
```

```
 const t0 = performance.now() + delay;
```

```
 function tick() {
```

```
 const elapsed = performance.now() - t0;
```

```
 const t = Math.min(elapsed / duration, 1);
```

```
 onUpdate(start + (end - start) * ease(t));
```

```
 if (t < 1) requestAnimationFrame(tick);
```

```
 else if (onEnd) onEnd();
```

```
 }
```

```
 setTimeout(() => requestAnimationFrame(tick), delay);
```

```
}
```

```
const BorderGlow = ({
```

```
 children,
```

```
 className = "",
```

```

edgeSensitivity = 30,
glowColor = '40 80 80',
backgroundColor = '#120F17',
borderRadius = 28,
glowRadius = 40,
glowIntensity = 1.0,
coneSpread = 25,
animated = false,
colors = ['#c084fc', '#f472b6', '#38bdf8'],
fillOpacity = 0.5,
}) => {
 const cardRef = useRef(null);

 const getCenterOfElement = useCallback((el) => {
 const { width, height } = el.getBoundingClientRect();
 return [width / 2, height / 2];
 }, []);

 const getEdgeProximity = useCallback((el, x, y) => {
 const [cx, cy] = getCenterOfElement(el);
 const dx = x - cx;
 const dy = y - cy;
 let kx = Infinity;
 let ky = Infinity;
 if (dx !== 0) kx = cx / Math.abs(dx);
 if (dy !== 0) ky = cy / Math.abs(dy);
 return Math.min(Math.max(1 / Math.min(kx, ky), 0), 1);
 }, [getCenterOfElement]);

 const getCursorAngle = useCallback((el, x, y) => {
 const [cx, cy] = getCenterOfElement(el);

```

```
const dx = x - cx;
const dy = y - cy;
if (dx === 0 && dy === 0) return 0;
const radians = Math.atan2(dy, dx);
let degrees = radians * (180 / Math.PI) + 90;
if (degrees < 0) degrees += 360;
return degrees;
}, [getCenterOfElement]));
```

```
const handlePointerMove = useCallback((e) => {
 const card = cardRef.current;
 if (!card) return;
```

```
 const rect = card.getBoundingClientRect();
 const x = e.clientX - rect.left;
 const y = e.clientY - rect.top;
```

```
 const edge = getEdgeProximity(card, x, y);
 const angle = getCursorAngle(card, x, y);
```

```
 card.style.setProperty('--edge-proximity', `${(edge * 100).toFixed(3)}%`);
 card.style.setProperty('--cursor-angle', `${angle.toFixed(3)}deg`);
}, [getEdgeProximity, getCursorAngle]);
```

```
useEffect(() => {
 if (!animated || !cardRef.current) return;
 const card = cardRef.current;
 const angleStart = 110;
 const angleEnd = 465;
 card.classList.add('sweep-active');
 card.style.setProperty('--cursor-angle', `${angleStart}deg`);
```

```

 animateValue({ duration: 500, onUpdate: v => card.style.setProperty('--edge-proximity', v) });
 animateValue({ ease: easeInCubic, duration: 1500, end: 50, onUpdate: v => {
 card.style.setProperty('--cursor-angle', `${(angleEnd - angleStart) * (v / 100) + angleStart}deg`);
 }});
 animateValue({ ease: easeOutCubic, delay: 1500, duration: 2250, start: 50, end: 100, onUpdate: v
=> {
 card.style.setProperty('--cursor-angle', `${(angleEnd - angleStart) * (v / 100) + angleStart}deg`);
 }});
 animateValue({ ease: easeInCubic, delay: 2500, duration: 1500, start: 100, end: 0,
 onUpdate: v => card.style.setProperty('--edge-proximity', v),
 onEnd: () => card.classList.remove('sweep-active'),
 });
 }, [animated]);

```

```
const glowVars = buildGlowVars(glowColor, glowIntensity);
```

```

return (
 <div
 ref={cardRef}
 onPointerMove={handlePointerMove}
 className={`border-glow-card ${className}`}
 style={{
 '--card-bg': backgroundColor,
 '--edge-sensitivity': edgeSensitivity,
 '--border-radius': `${borderRadius}px`,
 '--glow-padding': `${glowRadius}px`,
 '--cone-spread': coneSpread,
 '--fill-opacity': fillOpacity,
 ...glowVars,
 ...buildGradientVars(colors),
 }}
 >

```

```

 }}
 >

 <div className="border-glow-inner">
 {children}
 </div>
 </div>
);
};

```

```
export default BorderGlow;
```

```
...
```

```
Component CSS
```

```

``css

.border-glow-card {
 --edge-proximity: 0;
 --cursor-angle: 45deg;
 --edge-sensitivity: 30;
 --color-sensitivity: calc(var(--edge-sensitivity) + 20);
 --border-radius: 28px;
 --glow-padding: 40px;
 --cone-spread: 25;

 position: relative;
 border-radius: var(--border-radius);
 isolation: isolate;
 transform: translate3d(0, 0, 0.01px);
 display: grid;
 border: 1px solid rgb(255 255 255 / 15%);

```

```

background: var(--card-bg, #120F17);
overflow: visible;
box-shadow:
 rgba(0, 0, 0, 0.1) 0px 1px 2px,
 rgba(0, 0, 0, 0.1) 0px 2px 4px,
 rgba(0, 0, 0, 0.1) 0px 4px 8px,
 rgba(0, 0, 0, 0.1) 0px 8px 16px,
 rgba(0, 0, 0, 0.1) 0px 16px 32px,
 rgba(0, 0, 0, 0.1) 0px 32px 64px;
}

.border-glow-card::before,
.border-glow-card::after,
.border-glow-card > .edge-light {
 content: "";
 position: absolute;
 inset: 0;
 border-radius: inherit;
 transition: opacity 0.25s ease-out;
 z-index: -1;
}

.border-glow-card:not(:hover):not(.sweep-active)::before,
.border-glow-card:not(:hover):not(.sweep-active)::after,
.border-glow-card:not(:hover):not(.sweep-active) > .edge-light {
 opacity: 0;
 transition: opacity 0.75s ease-in-out;
}

/* colored mesh-gradient border */
.border-glow-card::before {

```

border: 1px solid transparent;

background:

linear-gradient(var(--card-bg, #120F17) 0 100%) padding-box,

linear-gradient(rgb(255 255 255 / 0%) 0% 100%) border-box,

var(--gradient-one, radial-gradient(at 80% 55%, hsla(268, 100%, 76%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-two, radial-gradient(at 69% 34%, hsla(349, 100%, 74%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-three, radial-gradient(at 8% 6%, hsla(136, 100%, 78%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-four, radial-gradient(at 41% 38%, hsla(192, 100%, 64%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-five, radial-gradient(at 86% 85%, hsla(186, 100%, 74%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-six, radial-gradient(at 82% 18%, hsla(52, 100%, 65%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-seven, radial-gradient(at 51% 4%, hsla(12, 100%, 72%, 1) 0px, transparent 50%))  
border-box,

var(--gradient-base, linear-gradient(#c299ff 0 100%)) border-box;

opacity: calc((var(--edge-proximity) - var(--color-sensitivity)) / (100 - var(--color-sensitivity)));

mask-image:

conic-gradient(

from var(--cursor-angle) at center,

black calc(var(--cone-spread) \* 1%),

transparent calc((var(--cone-spread) + 15) \* 1%),

transparent calc((100 - var(--cone-spread) - 15) \* 1%),

black calc((100 - var(--cone-spread)) \* 1%)

);

}

/\* colored mesh-gradient background fill near edges \*/

```

.border-glow-card::after {
 border: 1px solid transparent;
 background:
 var(--gradient-one, radial-gradient(at 80% 55%, hsla(268, 100%, 76%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-two, radial-gradient(at 69% 34%, hsla(349, 100%, 74%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-three, radial-gradient(at 8% 6%, hsla(136, 100%, 78%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-four, radial-gradient(at 41% 38%, hsla(192, 100%, 64%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-five, radial-gradient(at 86% 85%, hsla(186, 100%, 74%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-six, radial-gradient(at 82% 18%, hsla(52, 100%, 65%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-seven, radial-gradient(at 51% 4%, hsla(12, 100%, 72%, 1) 0px, transparent 50%))
padding-box,
 var(--gradient-base, linear-gradient(#c299ff 0 100%)) padding-box;

 mask-image:
 linear-gradient(to bottom, black, black),
 radial-gradient(ellipse at 50% 50%, black 40%, transparent 65%),
 radial-gradient(ellipse at 66% 66%, black 5%, transparent 40%),
 radial-gradient(ellipse at 33% 33%, black 5%, transparent 40%),
 radial-gradient(ellipse at 66% 33%, black 5%, transparent 40%),
 radial-gradient(ellipse at 33% 66%, black 5%, transparent 40%),
 conic-gradient(from var(--cursor-angle) at center, transparent 5%, black 15%, black 85%,
transparent 95%);

 mask-composite: subtract, add, add, add, add, add;

 opacity: calc(var(--fill-opacity, 0.5) * (var(--edge-proximity) - var(--color-sensitivity))) / (100 - var(--
color-sensitivity));

 mix-blend-mode: soft-light;
}

```

```

/* outer glow layer */
.border-glow-card > .edge-light {
 inset: calc(var(--glow-padding) * -1);
 pointer-events: none;
 z-index: 1;

 mask-image:
 conic-gradient(
 from var(--cursor-angle) at center, black 2.5%, transparent 10%, transparent 90%, black 97.5%
);

 opacity: calc((var(--edge-proximity) - var(--edge-sensitivity)) / (100 - var(--edge-sensitivity)));
 mix-blend-mode: plus-lighter;
}

.border-glow-card > .edge-light::before {
 content: "";
 position: absolute;
 inset: var(--glow-padding);
 border-radius: inherit;
 box-shadow:
 inset 0 0 0 1px var(--glow-color, hsl(40deg 80% 80% / 100%)),
 inset 0 0 1px 0 var(--glow-color-60, hsl(40deg 80% 80% / 60%)),
 inset 0 0 3px 0 var(--glow-color-50, hsl(40deg 80% 80% / 50%)),
 inset 0 0 6px 0 var(--glow-color-40, hsl(40deg 80% 80% / 40%)),
 inset 0 0 15px 0 var(--glow-color-30, hsl(40deg 80% 80% / 30%)),
 inset 0 0 25px 2px var(--glow-color-20, hsl(40deg 80% 80% / 20%)),
 inset 0 0 50px 2px var(--glow-color-10, hsl(40deg 80% 80% / 10%)),
 0 0 1px 0 var(--glow-color-60, hsl(40deg 80% 80% / 60%)),
 0 0 3px 0 var(--glow-color-50, hsl(40deg 80% 80% / 50%)),

```

```
0 0 6px 0 var(--glow-color-40, hsl(40deg 80% 80% / 40%)),
0 0 15px 0 var(--glow-color-30, hsl(40deg 80% 80% / 30%)),
0 0 25px 2px var(--glow-color-20, hsl(40deg 80% 80% / 20%)),
0 0 50px 2px var(--glow-color-10, hsl(40deg 80% 80% / 10%));
}
```

```
.border-glow-inner {
 display: flex;
 flex-direction: column;
 position: relative;
 overflow: auto;
 z-index: 1;
}
```

...

### ### Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

## Integrate the `<GlassIcons />` component from React Bits

You are helping integrate an open-source React component into an existing application.

### Component: GlassIcons

### Variant: JavaScript + CSS

---

### ### Usage Example

```
```jsx
import GlassIcons from './GlassIcons'

// update with your own icons and colors
const items = [
  { icon: <FiFileText />, color: 'blue', label: 'Files' },
  { icon: <FiBook />, color: 'purple', label: 'Books' },
  { icon: <FiHeart />, color: 'red', label: 'Health' },
  { icon: <FiCloud />, color: 'indigo', label: 'Weather' },
  { icon: <FiEdit />, color: 'orange', label: 'Notes' },
  { icon: <FiBarChart2 />, color: 'green', label: 'Stats' },
];

<div style={{ height: '600px', position: 'relative' }}>
  <GlassIcons items={items} className="custom-class"/>
</div>
```
```

### ### Props

| Prop      | Type                  | Default | Description                                                                                                                                                 |
|-----------|-----------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| items     | GlassIconsItem[]   [] |         | Array of items to render. Each item should include: an icon (React.ReactElement), a color (string), a label (string), and an optional customClass (string). |
| className | string                | ""      | Optional additional CSS class(es) to be added to the container.                                                                                             |

### ### Full Component Source

```
```jsx
import './GlassIcons.css';
```

```
const gradientMapping = {
  blue: 'linear-gradient(hsl(223, 90%, 50%), hsl(208, 90%, 50%))',
  purple: 'linear-gradient(hsl(283, 90%, 50%), hsl(268, 90%, 50%))',
  red: 'linear-gradient(hsl(3, 90%, 50%), hsl(348, 90%, 50%))',
  indigo: 'linear-gradient(hsl(253, 90%, 50%), hsl(238, 90%, 50%))',
  orange: 'linear-gradient(hsl(43, 90%, 50%), hsl(28, 90%, 50%))',
  green: 'linear-gradient(hsl(123, 90%, 40%), hsl(108, 90%, 40%))'
};
```

```
const GlassIcons = ({ items, className }) => {
  const getBackgroundStyle = color => {
    if (gradientMapping[color]) {
      return { background: gradientMapping[color] };
    }
    return { background: color };
  };
};
```

```
return (
  <div className={`icon-btns ${className} || ''`} >
    {items.map((item, index) => (
      <button key={index} className={`icon-btn ${item.customClass} || ''`} aria-label={item.label}
        type="button">
        <span className="icon-btn__back" style={getBackgroundStyle(item.color)}></span>
        <span className="icon-btn__front">
          <span className="icon-btn__icon" aria-hidden="true">
            {item.icon}
          </span>
        </span>
        <span className="icon-btn__label">{item.label}</span>
      </button>
    ))}
```

```
    </div>

  );
};

export default GlassIcons;

...

### Component CSS
```css
.icon-btns {
 display: grid;
 grid-gap: 5em;
 grid-template-columns: repeat(2, 1fr);
 margin: auto;
 padding: 3em 0;
 overflow: visible;
}

.icon-btn {
 background-color: transparent;
 outline: none;
 position: relative;
 width: 4.5em;
 height: 4.5em;
 perspective: 24em;
 transform-style: preserve-3d;
 -webkit-tap-highlight-color: transparent;
 border: none;
 cursor: pointer;
}
```

```
.icon-btn__back,
.icon-btn__front,
.icon-btn__label {
 transition:
 opacity 0.3s cubic-bezier(0.83, 0, 0.17, 1),
 transform 0.3s cubic-bezier(0.83, 0, 0.17, 1);
}
```

```
.icon-btn__back,
.icon-btn__front {
 border-radius: 1.25em;
 position: absolute;
 top: 0;
 left: 0;
 width: 100%;
 height: 100%;
}
```

```
.icon-btn__back {
 box-shadow: 0.5em -0.5em 0.75em hsla(223, 10%, 10%, 0.15);
 display: block;
 transform: rotate(15deg);
 transform-origin: 100% 100%;
 will-change: transform;
}
```

```
.icon-btn__front {
 background-color: hsla(0, 0%, 100%, 0.15);
 box-shadow: 0 0 0 0.1em hsla(0, 0%, 100%, 0.3) inset;
 backdrop-filter: blur(0.75em);
}
```

```
-webkit-backdrop-filter: blur(0.75em);
-moz-backdrop-filter: blur(0.75em);
display: flex;
transform-origin: 80% 50%;
will-change: transform;
}
```

```
.icon-btn__icon {
 margin: auto;
 width: 1.5em;
 height: 1.5em;
 display: flex;
 align-items: center;
 justify-content: center;
}
```

```
.icon-btn__label {
 font-size: 1em;
 white-space: nowrap;
 text-align: center;
 line-height: 2;
 opacity: 0;
 position: absolute;
 top: 100%;
 right: 0;
 left: 0;
 transform: translateY(0);
}
```

```
.icon-btn:focus-visible .icon-btn__back,
.icon-btn:hover .icon-btn__back {
```

```
 transform: rotate(25deg) translate3d(-0.5em, -0.5em, 0.5em);
 }
```

```
.icon-btn:focus-visible .icon-btn__front,
.icon-btn:hover .icon-btn__front {
 transform: translate3d(0, 0, 2em);
}
```

```
.icon-btn:focus-visible .icon-btn__label,
.icon-btn:hover .icon-btn__label {
 opacity: 1;
 transform: translateY(20%);
}
```

```
@media (min-width: 768px) {
 .icon-btns {
 grid-template-columns: repeat(3, 1fr);
 }
}
```

```
...
```

### ### Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.

## Integrate the `<Stepper />` component from React Bits

You are helping integrate an open-source React component into an existing application.

### Component: Stepper

### Variant: JavaScript + CSS

### Dependencies: motion

---

### Usage Example

```
```jsx
```

```
import Stepper, { Step } from './Stepper';
```

```
<Stepper
```

```
  initialStep={1}
```

```
  onStepChange={(step) => {
```

```
    console.log(step);
```

```
  }}
```

```
  onFinalStepCompleted={() => console.log("All steps completed!")}
```

```
  backButtonText="Previous"
```

```
  nextButtonText="Next"
```

```
>
```

```
<Step>
```

```
  <h2>Welcome to the React Bits stepper!</h2>
```

```
  <p>Check out the next step!</p>
```

```
</Step>
```

```
<Step>
```

```
  <h2>Step 2</h2>
```

```
  
```

```
  <p>Custom step content!</p>
```

```
</Step>
```

```

<Step>

  <h2>How about an input?</h2>

  <input value={name} onChange={(e) => setName(e.target.value)} placeholder="Your name?" />
</Step>

<Step>

  <h2>Final Step</h2>

  <p>You made it!</p>
</Step>
</Stepper>
...

```

Props

Prop	Type	Default	Description
children	ReactNode	—	The Step components (or any custom content) rendered inside the stepper.
initialStep	number	1	The first step to display when the stepper is initialized.
onStepChange	(step: number) => void	() => {}	Callback fired whenever the step changes.
onFinalStepCompleted	() => void	() => {}	Callback fired when the stepper completes its final step.
stepCircleContainerClassName	string	—	Custom class name for the container holding the step indicators.
stepContainerClassName	string	—	Custom class name for the row holding the step circles/connectors.
contentClassName	string	—	Custom class name for the step's main content container.
footerClassName	string	—	Custom class name for the footer area containing navigation buttons.
backButtonProps	object	{}	Extra props passed to the Back button.
nextButtonProps	object	{}	Extra props passed to the Next/Complete button.
backButtonText	string	"Back"	Text for the Back button.
nextButtonText	string	"Continue"	Text for the Next button when not on the last step.
disableStepIndicators	boolean	false	Disables click interaction on step indicators.
renderStepIndicator	{}	undefined	Renders a custom step indicator.

Full Component Source

```
``jsx

import React, { useState, Children, useRef, useEffect } from 'react';
import { motion, AnimatePresence } from 'motion/react';

import './Stepper.css';

export default function Stepper({
  children,
  initialStep = 1,
  onStepChange = () => {},
  onFinalStepCompleted = () => {},
  stepCircleContainerClassName = "",
  stepContainerClassName = "",
  contentClassName = "",
  footerClassName = "",
  backButtonProps = {},
  nextButtonProps = {},
  backButtonText = 'Back',
  nextButtonText = 'Continue',
  disableStepIndicators = false,
  renderStepIndicator,
  ...rest
}) {
  const [currentStep, setCurrentStep] = useState(initialStep);
  const [direction, setDirection] = useState(0);
  const stepsArray = Children.toArray(children);
  const totalSteps = stepsArray.length;
  const isCompleted = currentStep > totalSteps;
  const isLastStep = currentStep === totalSteps;
```

```
const updateStep = newStep => {  
  setCurrentStep(newStep);  
  if (newStep > totalSteps) {  
    onFinalStepCompleted();  
  } else {  
    onStepChange(newStep);  
  }  
};
```

```
const handleBack = () => {  
  if (currentStep > 1) {  
    setDirection(-1);  
    updateStep(currentStep - 1);  
  }  
};
```

```
const handleNext = () => {  
  if (!isLastStep) {  
    setDirection(1);  
    updateStep(currentStep + 1);  
  }  
};
```

```
const handleComplete = () => {  
  setDirection(1);  
  updateStep(totalSteps + 1);  
};
```

```
return (  
  <div className="outer-container" {...rest}>
```

```
<div className={`step-circle-container ${stepCircleContainerClassName}`} style={{ border: '1px solid #222' }}>
```

```
<div className={`step-indicator-row ${stepContainerClassName}`}>
```

```
{stepsArray.map((_, index) => {
```

```
  const stepNumber = index + 1;
```

```
  const isNotLastStep = index < totalSteps - 1;
```

```
  return (
```

```
    <React.Fragment key={stepNumber}>
```

```
      {renderStepIndicator ? (
```

```
        renderStepIndicator({
```

```
          step: stepNumber,
```

```
          currentStep,
```

```
          onStepClick: clicked => {
```

```
            setDirection(clicked > currentStep ? 1 : -1);
```

```
            updateStep(clicked);
```

```
          }
```

```
        })
```

```
      ) : (
```

```
        <StepIndicator
```

```
          step={stepNumber}
```

```
          disableStepIndicators={disableStepIndicators}
```

```
          currentStep={currentStep}
```

```
          onClickStep={clicked => {
```

```
            setDirection(clicked > currentStep ? 1 : -1);
```

```
            updateStep(clicked);
```

```
          }}
```

```
        />
```

```
      )}
```

```
      {isNotLastStep && <StepConnector isComplete={currentStep > stepNumber} />}
```

```
    </React.Fragment>
```

```
  );
```

```

    }}
  </div>

  <StepContentWrapper
    isCompleted={isCompleted}
    currentStep={currentStep}
    direction={direction}
    className={`step-content-default ${contentClassName}`}
  >
    {stepsArray[currentStep - 1]}
  </StepContentWrapper>

  {!isCompleted && (
    <div className={`footer-container ${footerClassName}`}>
      <div className={`footer-nav ${currentStep !== 1 ? 'spread' : 'end'}`}>
        {currentStep !== 1 && (
          <button
            onClick={handleBack}
            className={`back-button ${currentStep === 1 ? 'inactive' : ''}`}
            {...backButtonProps}
          >
            {backButtonText}
          </button>
        )}
        <button onClick={isLastStep ? handleComplete : handleNext} className="next-button"
        {...nextButtonProps}>
          {isLastStep ? 'Complete' : nextButtonText}
        </button>
      </div>
    </div>
  )}

```

```

    </div>
  </div>
);
}

```

```

function StepContentWrapper({ isCompleted, currentStep, direction, children, className }) {
  const [parentHeight, setParentHeight] = useState(0);

  return (
    <motion.div
      className={className}
      style={{ position: 'relative', overflow: 'hidden' }}
      animate={{ height: isCompleted ? 0 : parentHeight }}
      transition={{ type: 'spring', duration: 0.4 }}
    >
      <AnimatePresence initial={false} mode="sync" custom={direction}>
        {!isCompleted && (
          <SlideTransition key={currentStep} direction={direction} onHeightReady={h =>
setParentHeight(h)}>
            {children}
          </SlideTransition>
        )}
      </AnimatePresence>
    </motion.div>
  );
}

```

```

function SlideTransition({ children, direction, onHeightReady }) {
  const containerRef = useRef(null);

  useLayoutEffect(() => {

```

```

    if (containerRef.current) onHeightReady(containerRef.current.offsetHeight);
  }, [children, onHeightReady]);

  return (
    <motion.div
      ref={containerRef}
      custom={direction}
      variants={stepVariants}
      initial="enter"
      animate="center"
      exit="exit"
      transition={{ duration: 0.4 }}
      style={{ position: 'absolute', left: 0, right: 0, top: 0 }}
    >
      {children}
    </motion.div>
  );
}

```

```

const stepVariants = {
  enter: dir => ({
    x: dir >= 0 ? '-100%' : '100%',
    opacity: 0
  }),
  center: {
    x: '0%',
    opacity: 1
  },
  exit: dir => ({
    x: dir >= 0 ? '50%' : '-50%',
    opacity: 0
  })
}

```

```

    })
  };

export function Step({ children }) {
  return <div className="step-default">{children}</div>;
}

function StepIndicator({ step, currentStep, onClickStep, disableStepIndicators }) {
  const status = currentStep === step ? 'active' : currentStep < step ? 'inactive' : 'complete';

  const handleClick = () => {
    if (step !== currentStep && !disableStepIndicators) onClickStep(step);
  };

  return (
    <motion.div onClick={handleClick} className="step-indicator" style={disableStepIndicators ? {
      pointerEvents: 'none', opacity: 0.5 } : {}} animate={status} initial={false}>
      <motion.div
        variants={{
          inactive: { scale: 1, backgroundColor: '#222', color: '#a3a3a3' },
          active: { scale: 1, backgroundColor: '#5227FF', color: '#5227FF' },
          complete: { scale: 1, backgroundColor: '#5227FF', color: '#3b82f6' }
        }}
        transition={{ duration: 0.3 }}
        className="step-indicator-inner"
      >
        {status === 'complete' ? (
          <CheckIcon className="check-icon" />
        ) : status === 'active' ? (
          <div className="active-dot" />
        ) : (

```

```

        <span className="step-number">{step}</span>
      )}
    </motion.div>
  </motion.div>
);
}

```

```

function StepConnector({ isComplete }) {
  const lineVariants = {
    incomplete: { width: 0, backgroundColor: 'transparent' },
    complete: { width: '100%', backgroundColor: '#5227FF' }
  };

```

```

  return (
    <div className="step-connector">
      <motion.div
        className="step-connector-inner"
        variants={lineVariants}
        initial={false}
        animate={isComplete ? 'complete' : 'incomplete'}
        transition={{ duration: 0.4 }}
      />
    </div>
  );
}

```

```

function CheckIcon(props) {
  return (
    <svg {...props} fill="none" stroke="currentColor" strokeWidth={2} viewBox="0 0 24 24">
      <motion.path
        initial={{ pathLength: 0 }}

```

```
    animate={{ pathLength: 1 }}
    transition={{ delay: 0.1, type: 'tween', ease: 'easeOut', duration: 0.3 }}
    strokeLinecap="round"
    strokeLinejoin="round"
    d="M5 13l4 4L19 7"
  />
</svg>
);
}
```

...

Component CSS

```
```css
.outer-container {
 display: flex;
 min-height: 100%;
 flex: 1 1 0%;
 flex-direction: column;
 align-items: center;
 justify-content: center;
 padding: 1rem;
}
```

```
@media (min-width: 640px) {
 .outer-container {
 aspect-ratio: 4 / 3;
 }
}
```

```
@media (min-width: 768px) {
```

```
.outer-container {
 aspect-ratio: 2 / 1;
}
}
```

```
.step-circle-container {
 margin-left: auto;
 margin-right: auto;
 width: 100%;
 max-width: 28rem;
 border-radius: 2rem;
 box-shadow:
 0 20px 25px -5px rgba(0, 0, 0, 0.1),
 0 10px 10px -5px rgba(0, 0, 0, 0.04);
}
```

```
.step-indicator-row {
 display: flex;
 width: 100%;
 align-items: center;
 padding: 2rem;
}
```

```
.step-content-default {
 position: relative;
 overflow: hidden;
}
```

```
.step-default {
 padding-left: 2rem;
 padding-right: 2rem;
```

```
}
```

```
.footer-container {
 padding-left: 2rem;
 padding-right: 2rem;
 padding-bottom: 2rem;
}
```

```
.footer-nav {
 margin-top: 2.5rem;
 display: flex;
}
```

```
.footer-nav.spread {
 justify-content: space-between;
}
```

```
.footer-nav.end {
 justify-content: flex-end;
}
```

```
.back-button {
 transition: all 350ms;
 border-radius: 0.25rem;
 padding: 0.25rem 0.5rem;
 color: #a3a3a3;
 cursor: pointer;
}
```

```
.back-button:hover {
 color: #52525b;
}
```

```
}
```

```
.back-button.inactive {
 pointer-events: none;
 opacity: 0.5;
 color: #a3a3a3;
}
```

```
.next-button {
 transition: all 350ms;
 display: flex;
 align-items: center;
 justify-content: center;
 border-radius: 9999px;
 background-color: #5227ff;
 color: #fff;
 font-weight: 500;
 letter-spacing: -0.025em;
 padding: 0.375rem 0.875rem;
 cursor: pointer;
}
```

```
.next-button:hover {
 background-color: #5227ff;
}
```

```
.next-button:active {
 background-color: #5227ff;
}
```

```
.step-indicator {
```

```
position: relative;
cursor: pointer;
outline: none;
}
```

```
.step-indicator-inner {
 display: flex;
 height: 2rem;
 width: 2rem;
 align-items: center;
 justify-content: center;
 border-radius: 9999px;
 font-weight: 600;
}
```

```
.active-dot {
 height: 0.75rem;
 width: 0.75rem;
 border-radius: 9999px;
 background-color: #fff;
}
```

```
.step-number {
 font-size: 0.875rem;
}
```

```
.step-connector {
 position: relative;
 margin-left: 0.5rem;
 margin-right: 0.5rem;
 height: 0.125rem;
```

```
flex: 1;

overflow: hidden;

border-radius: 0.25rem;

background-color: #52525b;
}
```

```
.step-connector-inner {

 position: absolute;

 left: 0;

 top: 0;

 height: 100%;
}
```

```
.check-icon {

 height: 1rem;

 width: 1rem;

 color: #fff;
}
```

...

### ### Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import the CSS file alongside the component.
4. Import and render the component using the usage example above as a starting point.
5. Adjust props as needed for the specific use case — refer to the props table for all available options.